SEMANA 7 — EXPORT E INFORMES DEL TICKET

ÍNDICE

Tarea 1 — Exportar un ticket a “Informe HTML” (1 clic)..... 2
Tarea 2 — Añadir “cabecera de informe” (branding + metadatos)..... 4
Tarea 3 — Export CSV del listado de tickets..... 5
Tarea 4 — Generar PDF del informe..... 7
Tarea 5 — Registrar exportaciones en auditoría..... 9
Conclusión..... 11

Tarea 1 — Exportar un ticket a “Informe HTML” (1 clic)

El objetivo de esta tarea es permitir a admins y técnicos generar un informe en formato HTML de un ticket, que incluya todos los datos relevantes: información del ticket, comentarios, historial de cambios y lista de adjuntos. Este informe debe ser accesible desde el detalle del ticket mediante un botón "**Exportar informe**" y estar protegido por permisos (solo admin/técnico).

Para la parte lógica se creó el archivo **public\export_ticket.php** en el cual se hace varios procesos que se han hecho varias veces a lo largo del proyecto, como autentificar que el usuario no sea lector, obtiene el ID del ticket a través de un GET, hace un par de consultas sobre los datos del ticket, los comentarios, los adjuntos y el historial de cambios,... Y por último carga la vista de la exportación, creando un HTML que se muestra a parte.

```php
session_start();

// Verificar autenticación y rol
if (!isset($_SESSION['user_id']) || $_SESSION['role'] === 'lector') {
    die("No autorizado.");
}

require_once __DIR__ . '/../app/config.php';
require_once __DIR__ . '/../app/ticket_constants.php';

$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if (!$id) {
    die("ID de ticket inválido.");
}

// Obtener datos del ticket
$stmt = $db->prepare("
    SELECT t.*, u.username AS assigned_user
    FROM tickets t
    LEFT JOIN users u ON t.assigned_to = u.id
    WHERE t.id = :id
");
$stmt->execute([':id' => $id]);
$ticket = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$ticket) {
    die("Ticket no encontrado.");
}

```

Para la vista se formó el archivo **views\export_ticket.php** que es un documento HTML autocontenido con estilos CSS básicos para que sea legible tanto en pantalla como en impresión, a continuación un ejemplo con el Ticket 3.

Título: Ticket 3
Estado: En_proceso
Prioridad: Baja
Asignado a: Sin asignar
Creado: 2026-02-26 10:32:31
Última actualización: 2026-03-03 11:51:43
Descripción:
Esto es una prueba para ver la zona horaria

Comentarios

No hay comentarios.

Historial de cambios

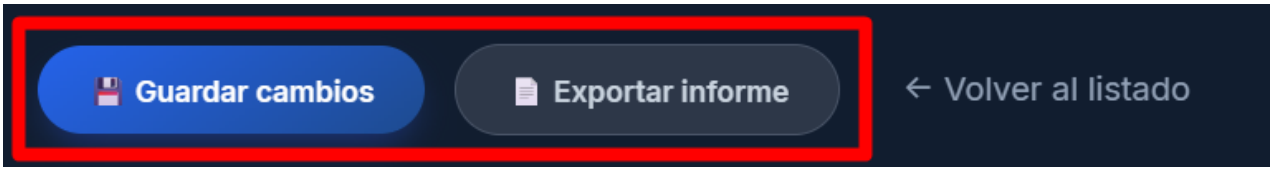
admin (2026-03-03 11:51:43)

Cambió **status** de nuevo a en_proceso

Adjuntos

Semana 5 — Auditoría completa + seguridad anti “errores típicos”.pdf (766.32 KB) - Subido por admin el 2026-03-03 11:50:02

En **views/tickets/detail.php** se añadió un botón "**Exportar informe**" junto a las acciones de edición (guardar cambios, volver). Solo se muestra si el usuario no es lector.



Los botones marcados con el recuadro rojo, en caso de ser lector, no se muestran. Lógicamente, como antes se mencionó en la parte lógica el bloqueo no solo se hace visualmente si no que también a través del controlador, al intentar entrar con el enlace da un error y nos dice que no tenemos permisos para entrar como lector.

No autorizado.

Tarea 2 — Añadir “cabecera de informe” (branding + metadatos)

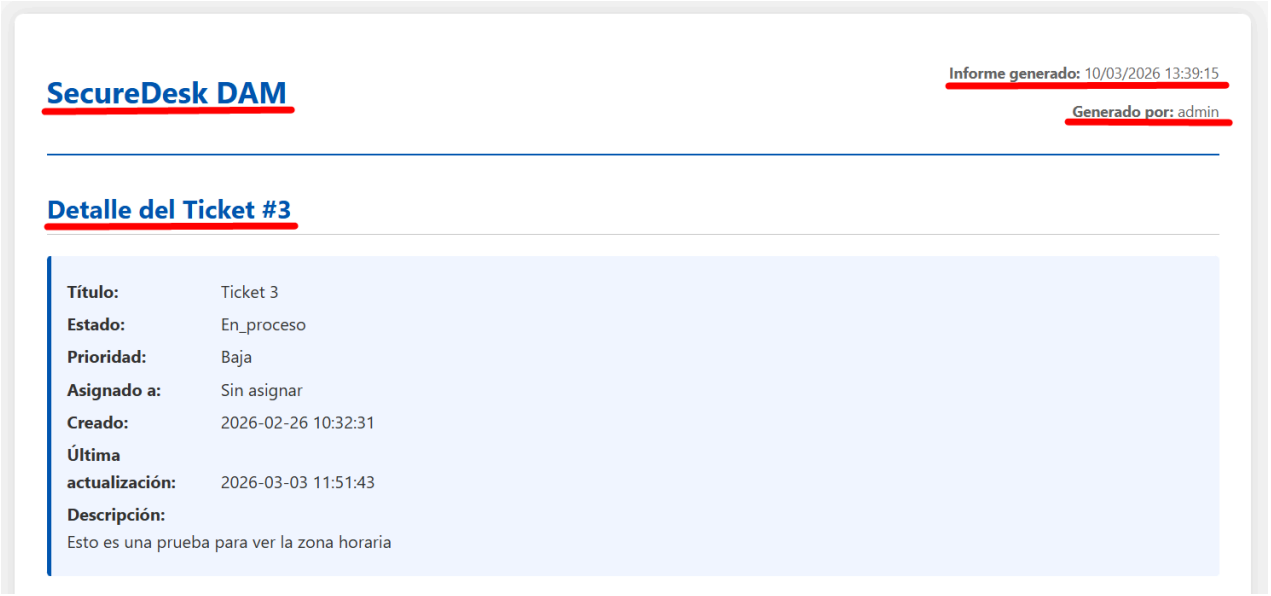
En esta tarea se ha mejorado el informe sobre el informe, se ha añadido una cabecera donde se encuentran los metadatos como el creador del informe, fecha y hora, id del ticket y un título de “SecureDesk”.

Lo primero que se ha modificado es la parte lógica para obtener el usuario que genera el informe.

```
// Obtener nombre del usuario que genera el informe
$usuario_informe = $_SESSION['username'] ?? 'Desconocido';
```

Y lo siguiente en modificarse es la vista, se rediseñó la cabecera del informe para que incluya:

- Título con el nombre de la aplicación.
- Metadatos en una tabla o lista horizontal.
- ID del ticket destacado.



Se mejoró también el estilo del cuadro donde se muestra la información del ticket, dándole un borde en el lado izquierdo azul marino y un fondo azul claro.

Tarea 3 — Export CSV del listado de tickets

En esta tarea se trata de hacer un botón el cual haga un documento como un excel es decir, en formato **.csv** en el cual se muestran los detalles de los tickets.

Comenzamos por hacer la parte lógica de exportar los tickets **public/export_tickets_csv** este script recibe los parámetros de filtro (opcionales) por **GET**, consulta la base de datos y genera un archivo CSV descargable.

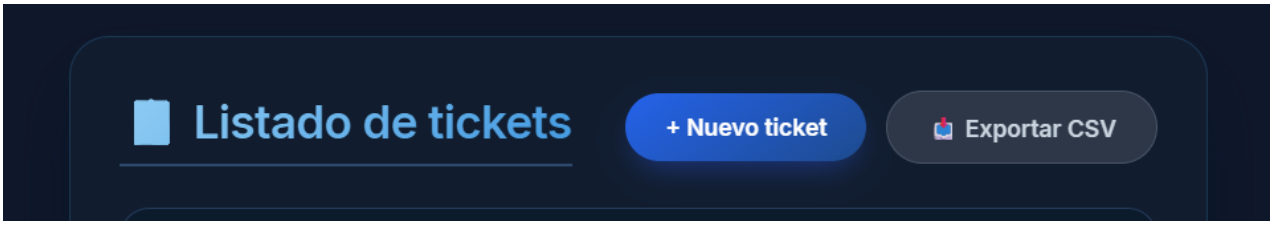
A continuación explicamos el código y como se ha hecho el script:

1. Se identifica y verifica que el usuario se encuentra autenticado
2. En cuanto a los filtros se toman los mismos parámetros de **status** y **priority** que se usan en la vista de listado. Si no se proporcionan, se exportan todos los tickets.
3. Se obtienen los campos requeridos y se usa **COALESCE** para mostrar "**Sin asignar**" cuando no hay usuario asignado.
4. Y por último la generación del CSV (que tambien mostramos una imagen con el código)
   - 4.1. Se usa **header()** para indicar que es un archivo CSV descargable.
   - 4.2. Se añade el **BOM (\xEF\xBB\xBF)** al principio para que Excel reconozca la codificación UTF-8 (evita caracteres extraños como Ã). Ya que en las pruebas se generó un error debido a dicho problema e investigando logré arreglarlo con dicha librería.
   - 4.3. Se emplea **fputcsv()** con delimitador ; (punto y coma) y encierro " (comillas dobles), que es el formato esperado por Excel en configuraciones regionales de habla hispana. Todo esto lo he tenido que investigar por mi cuenta tras fallos y fallos y ayudándome de la IA para que me explicase cómo funcionaba y entenderlo más a fondo.
   - 4.4. El nombre del archivo incluye una marca de tiempo para que sea único y saber en qué día se generó el csv.

	A	B	C	D	E	F
1	ID	Título	Estado	Prioridad	Asignado a	Fecha de creación
2	3	Ticket 3	en_proceso	baja	Sin asignar	26/02/2026 10:32
3	2	Ticket 2	en_proceso	media	tecnico1	24/02/2026 10:57
4	1	Ticket 1	nuevo	alta	Sin asignar	16/02/2026 11:37

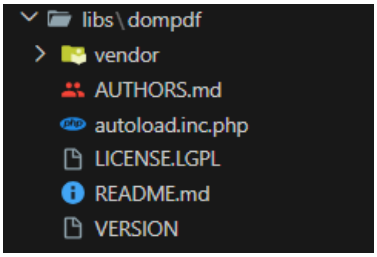
```php
// ============================================
// Configurar cabeceras para descargar CSV
// ============================================
$filename = 'tickets_' . date('Y-m-d_H-i-s') . '.csv';

header('Content-Type: text/csv; charset=utf-8');
header('Content-Disposition: attachment; filename="' . $filename . '"');

// Añadir BOM (Byte Order Mark) para que Excel interprete UTF-8 correctamente
echo "\xEF\xBB\xBF";

// Abrir salida como archivo
$output = fopen('php://output', 'w');

// Escribir la fila de cabeceras con separador punto y coma
fputcsv($output, ['ID', 'Título', 'Estado', 'Prioridad', 'Asignado a', 'Fecha de creación'], ';', '"');

// Escribir cada ticket
foreach ($tickets as $ticket) {
    fputcsv($output, [
        $ticket['id'],
        $ticket['title'],
        $ticket['status'],
        $ticket['priority'],
        $ticket['assigned_to'],
        $ticket['created_at']
    ], ';', '"');
}
```

Por la vista fue algo muy simple, en **views\tickets\list.php** se añadió un botón que redirige a **public/export_tickets_csv** haciendo que se descargue el archivo, el botón se posicionó al lado de “+ Nuevo ticket” ya que creo que puede encajar bien, se mejoró y se puso de un color gris para evitar confusiones.



Tarea 4 — Generar PDF del informe

En esta tarea se trata de hacer un informe descargable en versión pdf donde se muestre la información de un ticket. Para generar PDF desde HTML se utilizó la librería **Dompdf**. Se instaló manualmente descargando el paquete y colocándolo en una nueva carpeta **libs/**.



Para generar el pdf se hizo un nuevo archivo **public\export_ticket_pdf.php** este controlador es muy similar al de exportación HTML (**public/export_ticket.php**), pero en lugar de mostrar la vista directamente, captura su salida y la convierte a **PDF**.

```php
// Generar el HTML del informe (reutilizamos la misma vista pero capturamos la salida)
ob_start();
include __DIR__ . '/../views/export_ticket.php';
$html = ob_get_clean();

// Configurar Dompdf
$options = new Options();
$options->set('isRemoteEnabled', false);
$options->set('defaultFont', 'Helvetica');
$dompdf = new Dompdf($options);
$dompdf->loadHtml($html);
$dompdf->setPaper('A4', 'portrait');
$dompdf->render();

// Enviar el PDF al navegador para descarga
$filename = 'ticket_' . $id . '_informe_' . date('Y-m-d') . '.pdf';
$dompdf->stream($filename, array('Attachment' => 1));
exit;
```

Al principio del documento se carga el encargado de pasar el HTML a pdf.

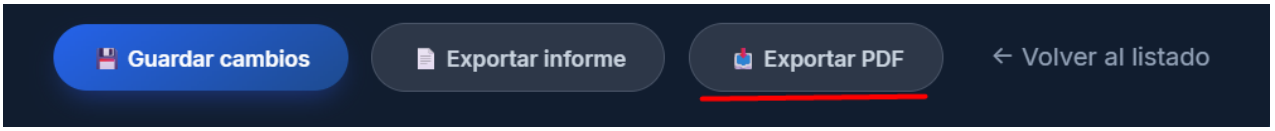
```php
// Cargar Dompdf
require_once __DIR__ . '/../libs/dompdf/vendor/autoload.php';

use Dompdf\Dompdf;
use Dompdf\Options;
```

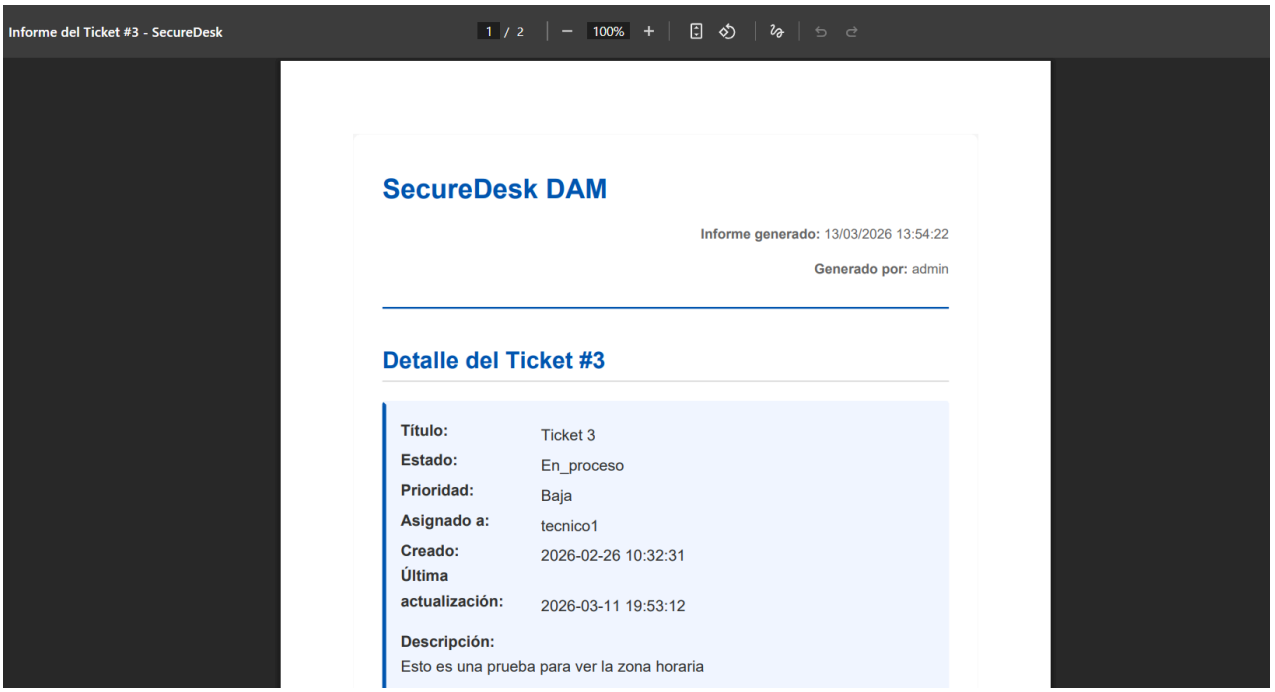
La explicación del código al completo es una copia de la exportación en HTML se obtienen los datos necesarios con las mismas consultas, hasta su parte final en la cual se captura el HTML y se pasa a pdf a través del comando **ob_start()** y **ob_get_clean()** se almacena el HTML generado por la vista en una variable sin enviarlo al navegador.

Tras esto, se hacen un par de configuraciones en Dompdf:

- **isRemoteEnabled** a **false** por seguridad (no se cargan imágenes externas).
- **defaultFont** a 'Helvetica' (fuente segura para PDF).
- Tamaño de papel A4 y orientación vertical.

Y por último **$dompdf->stream()** envía el PDF al navegador con **Attachment=1** para forzar la descarga (en lugar de mostrarlo en línea). Se carga la misma vista que en la tarea 1, haciendo que se vea igual, esta no ha necesitado cambios ninguno y en la vista de **views/tickets/detail.php** se añadió un nuevo botón junto al de "Exportar informe" que al nuevo le hemos llamado "**Descargar pdf**". A continuación una foto con el botón y tras esa imagen, una foto de como se muestra el **PDF**.





Tarea 5 — Registrar exportaciones en auditoría

En esta tarea se trata de registrar en la pestaña de auditoría cada una de las veces que se hace una exportación de los tickets. Esto permite a los administradores consultar posteriormente quién, cuándo y qué tipo de exportación se llevó a cabo.

Básicamente en cada uno de los controladores de exportación **public/export_ticket.php** (controlador HTML), **public/export_tickets_csv.php** (controlador CSV) y **public/export_ticket_pdf.php** (controlador PDF).

```
require_once __DIR__ . '/../app/audit.php';
```

Depende de el tipo de exportación se manda un mensaje u otro, empezando por **public/export_ticket.php** (controlador HTML):

```
// Registrar auditoría
if (function_exists('registrar_auditoria')) {
    $details = "Exportó informe HTML del ticket #$id";
    registrar_auditoria($db, $_SESSION['user_id'], 'EXPORT_HTML', 'ticket', $id, $details);
}
```

También se modificó **public/export_tickets_csv.php** (controlador CSV):

```
// Registrar auditoría
if (function_exists('registrar_auditoria')) {
    $filtros = [];
    if (!empty($_GET['status'])) $filtros[] = 'status=' . $_GET['status'];
    if (!empty($_GET['priority'])) $filtros[] = 'priority=' . $_GET['priority'];
    $desc_filtros = $filtros ? ' con filtros: ' . implode(', ', $filtros) : '';
    $details = "Exportó listado de tickets a CSV" . $desc_filtros;
    registrar_auditoria($db, $_SESSION['user_id'], 'EXPORT_CSV', 'ticket_list', null, $details);
}
```

Y por último **public/export_ticket_pdf.php** (controlador PDF):

```
// Registrar auditoría
if (function_exists('registrar_auditoria')) {
    $details = "Exportó informe PDF del ticket #$id";
    registrar_auditoria($db, $_SESSION['user_id'], 'EXPORT_PDF', 'ticket', $id, $details);
}
```

Gracias a estos cambios, en la ventana de las auditorías se muestran cuándo y por quien ha sido generado un informe dependiendo del formato:

FECHA	USUARIO	ACCIÓN	ENTIDAD	ID	DETALLE	IP
2026-03-13 13:59:06	admin	EXPORT_HTML	ticket	1	Exportó informe HTML del ticket #1	::1
2026-03-13 13:43:52	admin	EXPORT_PDF	ticket	3	Exportó informe PDF del ticket #3	::1
2026-03-13 13:43:31	admin	EXPORT_HTML	ticket	3	Exportó informe HTML del ticket #3	::1

Conclusión

Esta semana hemos dotado a Secure Desk de una funcionalidad clave para entornos profesionales: la capacidad de generar informes detallados de los tickets en varios formatos. Comenzamos implementando la exportación a HTML, creando un informe imprimible que incluye todos los datos del ticket, comentarios, historial de cambios y adjuntos. Posteriormente, añadimos la exportación a CSV del listado de tickets, respetando los filtros activos y solucionando problemas de codificación para que los archivos se abran correctamente en Excel. Como broche de oro, integramos la generación de PDF utilizando la librería Dompdf, ofreciendo un formato de descarga universal y de aspecto profesional.

Esta semana me he tenido que buscar bastante las habichuelas para poder hacer las exportaciones en CSV sin que excel me diese errores y también en los PDF para que los pueda generar a través de la librería, que he aprendido a través de un video de youtube, una semana bastante interesante.

A continuación dejo el código en GitHub como siempre:
https://github.com/elgancho06/Practicas-DAM